

AMATH 582: HOMEWORK 2

JADE GLAISTER

Applied Math, University of Washington, Seattle, WA
jglais@uw.edu

ABSTRACT. This report describes the process of designing an algorithm to characterize the movements of a robot, OptimuS-VD, in real-time of 3 movements: jumping, running, and walking. The algorithm uses Principle Component Analysis as an unsupervised learning method to reduce the dimensions of the data. Then, centroid-based classification is used to classify the activity for each sample dataset. Trained data is used to train the Principle Component analysis model, then test data is projected onto the k-space created. The accuracies of this model on classifying both the trained and test data are calculated, showing this model has 100% accuracy for an optimal number of spatial nodes of 7. Additionally, K-Nearest Neighbors classification is also used to classify the samples, and this method had 100% accuracy with 2 spatial nodes while using 1 nearest neighbor point.

1. INTRODUCTION AND OVERVIEW

This report describes the process of designing an algorithm to characterize the movements of a robot, OptimuS-VD, in real-time. Data were provided of recorded movements of 38 of the robot's joints at 60 Hz, collected as Euler angles, which can be transformed to xyz coordinates. The samples were recorded for 1.4 seconds, causing each sample to be represented as a matrix comparing the 114 features (due to three different axes of 38 data points) to the 100 time steps. Five samples were taken of 3 different movements: walking, jumping, and running. The goal of this process is to build a projection of the recordings to a lower dimension and then the number of coordinates to visualize the movements. So, Principle Component Analysis (PCA), a supervised learning concept, was used for this reduction and projection. The algorithm that determined the activity of a sample dataset was designed with unsupervised learning as centroids of each activity were found using the provided activity type of each training sample. Minimal distance of a dataset from each activity's centroid classified the activity type of a sample. The provided sample datasets were used to train the model, and additional provided datasets tested the model. The accuracies of these classifications evaluated the performance of this model.

2. THEORETICAL BACKGROUND

A large part of this model depends on Principle Component Analysis (PCA) to reduce the dimensions of the datasets. The goal of Principle Component Analysis is to find a sequence of orthonormal basis functions so that the first term gives the best 1-term approximation of the function, the first and second gives the best 2-term approximation, and so on. These orthonormal functions are called the Proper Orthogonal Modes (POD). The following formula demonstrates the POD expansion of a function, where $f(x, t)$ is the original function, N is the total number of modes, $a_j(t)$ determines the weights of the spatial nodes, and $\phi_j(x)$ are the modes [4].

$$f(x, t) \approx \sum_{j=1}^N a_j(t) \phi_j(x)$$

Singular Value Decomposition (SVD) is the basis of this method because its outcome "orders singular values and associated orthonormal basis functions that capture as much energy as possible." SVD works through the following formula, where U and V are orthonormal bases and Σ contains the singular values of the matrix A [4].

$$A = U\Sigma V^*$$

The main method in this report uses supervised learning by comparing the distance between the dataset and the classification centroids, commonly called centroid-based classification. This method takes training data and defines a centroid for each classification type. Classification is established by evaluating the distance between test data points and the centroid. Centroid-based classification has been used for a variety of applications, including many movement-based application. One research paper used this method to recognize people based on features of their walking form, and they had a high classification accuracy of 98% for the favorable aspect angle, showing high performance of this method [1].

3. ALGORITHM IMPLEMENTATION AND DEVELOPMENT

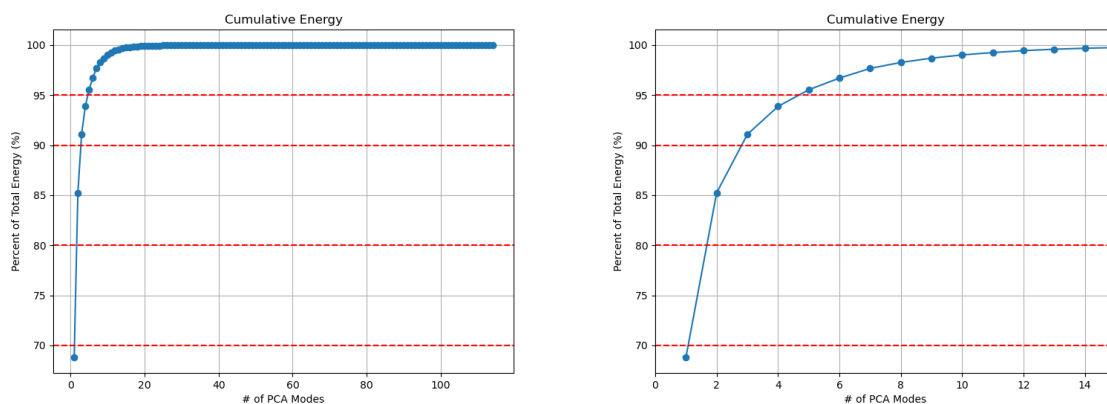
The primary function used to analyze this dataset is Scikit Learn's PCA Decomposition. This function uses SVD to project data to a lower space and centers the data [5]. Scikit Learn's Metrics was also used to calculate the accuracy score of the trained and test data [5]. The last Scikit Learn package used was their Neighbors package, which was used to perform K-Nearest Neighbors classification [6]. This function the k-nearest neighbors vote to classify data by comparing the dataset to its closest neighbors. Additionally, NumPy was used to manage matrices [2]. To plot everything, Matplotlib's PyPlot was used [3].

4. COMPUTATIONAL RESULTS

A training parameter was entered into the PCA algorithm that contained all the sample data. The shape of this parameter was 114 x 1500, maintaining the convention that the rows represent the features and spatial variables while the columns corresponded to the samples and time steps. This training parameter was transposed and entered into PCA to truncate the matrix. To determine the number of k spatial nodes needed to maintain adequate cumulative energy to represent the data, a range of 1 to 114 spatial nodes were entered into the PCA algorithm. The cumulative energy was plotted, and the full plot is shown in Figure 1a, and a zoomed in plot is shown in Figure 1b. As shown in the plot, 70% energy is achieved with about 1 node, 80% energy after 2 nodes, 90% energy after 3 nodes, and 95% after 5 nodes.

The first 2 principle components were plotted against each other to show how they display the data of different activities. By doing this, the orthogonal directions of maximum variance and second maximum variances are plotted against each other, showing the movement of each activity. This is shown in Figure 2.

To show how adding another principle component affects how the data is displayed, the first 3 principle components were also plotted against each other. This plot is shown in Figure 3.



(A) Full Plot of the number PCA Modes vs. Cumulative Energy (B) Zoomed-In Plot of the number of PCA Modes (0-15) vs. Cumulative Energy

FIGURE 1. These plots show the relationship between the number of PCA Modes and cumulative energy, demonstrating the amount of energy of the full size matrix was kept as PCA reduced the matrix to different K numbers of PCA modes. There are horizontal dashed lines at 70%, 80%, 90%, and 95% to assist with finding the number of PCA modes that achieve these energy percentages.

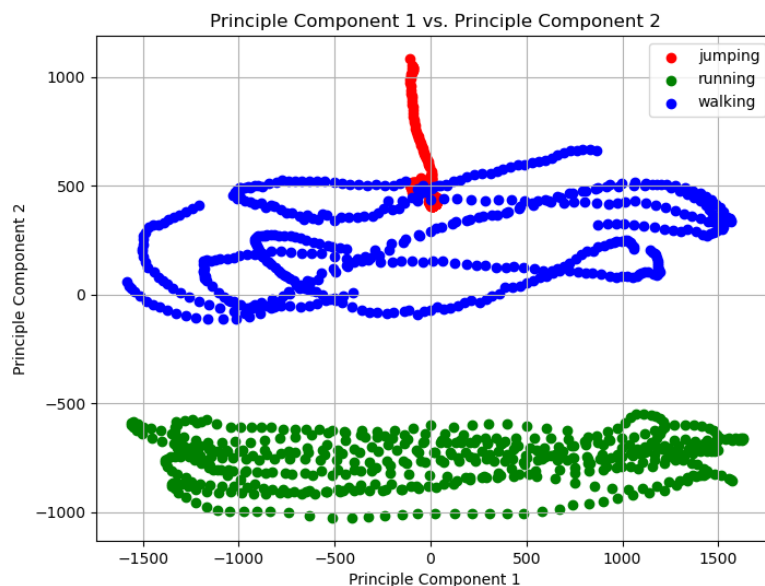


FIGURE 2. This plot shows the movements of each activity (jumping, running, walking) in 2 dimensions by plotting Principle Component 1 (the direction with the largest variance) vs. Principle Component 2 (the direction with the second largest variance). The result shows 3 distinct movements between the 3 activities.

Principle Component 1 vs. Principle Component 2 vs. Principle Component 3

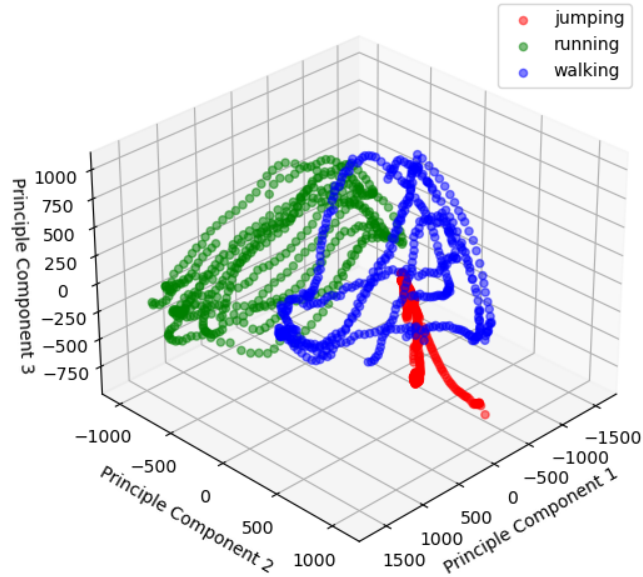


FIGURE 3. This plot shows the movements of each activity (jumping, running, walking) in 3 dimensions by plotting Principle Component 1 (the direction with the largest variance) vs. Principle Component 2 (the direction with the second largest variance) vs. Principle Component 3 (the direction with the third largest variance). The result shows 3 distinct movements between the three activities.

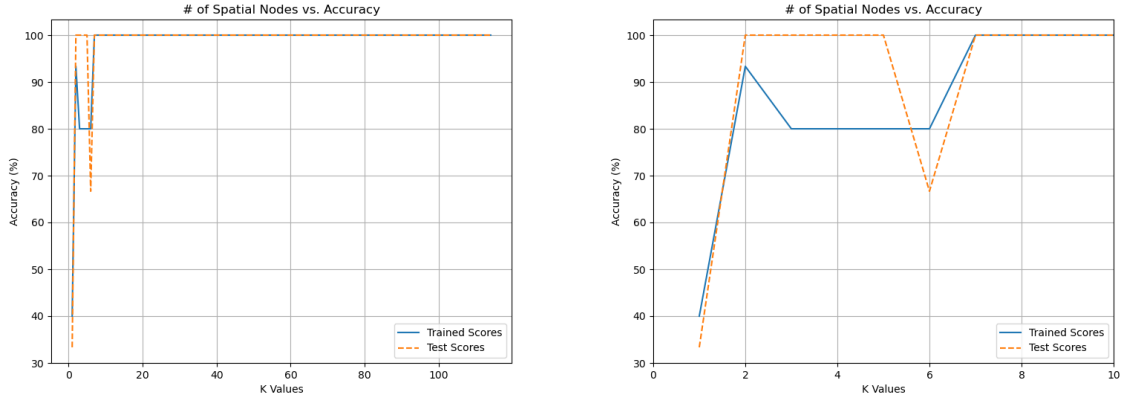
To create a model that identifies the sample's activity with only the dataset, a vector of ground truth labels was established to calculate the centroid for each activity. Each activity's centroid was calculated as the mean of the 5 PCA-applied sample datasets for each activity. The mean was taken for each principle component direction, so each centroid had k -dimensions. For example, the centroid when the number of spatial nodes (k) was three is shown in Table 1.

Activity	PC1	PC2	PC3
Jumping	-23.89	499.37	-72.50
Running	60.77	-752.72	-103.41
Walking	-36.88	253.35	175.91

TABLE 1. This table displays the coordinates of the centroid for each activity in a 3-D principal component coordinate system when the number of spatial nodes (k) is 3.

For each sample dataset, each activity's distance was found for each activity as the Frobenius norm of a $(100 \times k)$ matrix composed of the distances between each time point and the activity's centroid. The minimal activity distance classified the sample dataset as that activity such as if the distance between the dataset and the 'jumping' activity's centroid was smaller than that of the other two activities, the model classified that sample as a 'jumping' sample. The model's activity

classifications of the training data were compared to their known activity classifications as an accuracy percentage. This method was used on the testing sample datasets as they were projected onto the same k-space as the centroids and compared the same way to determine the activity of each testing sample. The model's activity classifications of the test data were also compared to their known activity classifications as an accuracy percentage. Figure 4a has the plot of the relationship between the K values (number of spatial nodes) from 1 to 115 compared to the accuracy. Because both the trained and test scores remain at 100% past a K value of 7, Figure 4b shows a closer view of this relationship with K values from 1 to 10. The lowest K value where the accuracy of both the trained and test data are at 100% is 7, so I determined $K=7$ to be the optimal K value.



(A) Full Plot of the K Value vs. Accuracy

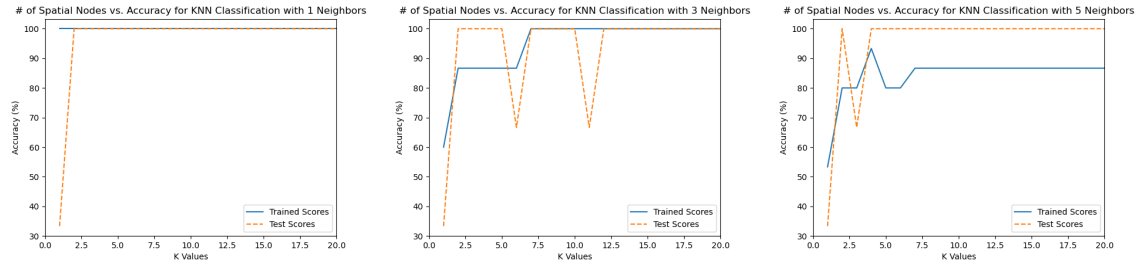
(B) Zoomed-In Plot of the K Value vs. Accuracy

FIGURE 4. These plots show the relationship between the K value (number of spatial nodes) and the accuracy the model's classification of the training samples' activities and the testing samples' activities. For K values of 2 to 5, the model for the testing samples is 100% accurate while the model is 80-93% accurate for the training samples. The test score dips at a K value of 6. Both the trained and test scores are 100% when K is greater than or equal to 7.

An alternative classifier, K-Nearest Neighbors (KNN) classifier, was also used on these samples and its performance was evaluated similarly. Instead of using centroids to determine the activity of each sample, the KNN classifier computes the distance between a test point and its K-nearest points. Because K already represents the number of spatial nodes, N is used for the number of nearest neighbors. N values of 1, 3, and 5 were tested and shown in Figures 5a, 5b, and 5c. These plots demonstrate that an N value of 1 is optimal for this data, likely because the data does not have a lot of noise and it is well-separated.

5. SUMMARY AND CONCLUSIONS

This report showed that a supervised model that depends on centroid distances paired with an unsupervised model that uses Principle Component Analysis can accurately classify the activities performed by OptimuS-VD in each sample. With an optimal number of spatial nodes of 7, this model is 100% accurate at classifying the trained data and the test data. Additionally, Principle Component Analysis can be paired with a different supervised model that uses K-Nearest Neighbors to accurately classify these activities in each sample. The optimal nearest neighbor value for this is 1 because the model is 100% accurate at classifying the trained and the test data with only 2 spatial nodes.



(A) K value vs. Accuracy for N=1 Neighbor (B) K value vs. Accuracy for N=3 Neighbors (C) K value vs. Accuracy for N=5 Neighbors

FIGURE 5. These plots show the relationship between the K value (number of spatial nodes) and the accuracy the KNN model's classification of the training samples' activities and the testing samples' activities for different N values (number of nearest neighbors compared). These plots are all zoomed in, showing K values 0 to 20. When K is greater than 20, the accuracy continues to be 100% until K=115.

To further extend this investigation, more test samples should be analyzed by these models. Each jumping, running, and walking sample all seemed relatively similar to other samples of the same activity, which made it easier for these models to accurately classify the data. Different styles of jumping, running, and walking should be provided to these models to ensure that they can accurately classify a larger variety of each activity.

ACKNOWLEDGEMENTS

The TA, Saba Heravi, provided a lot of assistance for the report through her office hours. This support included verifying matrix sizes and verifying graphs.

Additionally, I received help from many peers in this course, with regard to verifying graphs, debugging, and discussing how to use new methods: Anja Vogt, Anna Rutherford, Jonathan Elsner, Maddy Kovalski, Hailey Sparks, Jaxon Tuggle, Hunter Lybert, and John Murphy.

REFERENCES

- [1] F. Fioranelli, M. Ritchie, and H. Griffiths. Performance Analysis of Centroid and SVD Features for Personnel Recognition Using Multistatic Micro-Doppler. *IEEE Geoscience and Remote Sensing Letters*, 13(5):725–729, May 2016. Conference Name: IEEE Geoscience and Remote Sensing Letters.
- [2] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with numpy. *Nature*, 585:357–362, 2020.
- [3] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science Engineering*, 9(3):90–95, 2007.
- [4] J. N. Kutz. Data-Driven Modeling & Scientific Computation.
- [5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [6] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.